

NESTUP TECH

ArUco Marker-Based Wall Measurement System

Complete Business & Technical Implementation Plan
From Concept to Production

Version 1.0 | February 2026
Confidential

Table of Contents

Table of Contents.....	2
1. What is an ArUco Marker?	4
1.1 Technical Definition	4
1.2 Key Properties	4
1.3 Why ArUco for Nestup?	4
2. How the System Works	5
2.1 System Architecture Overview	5
2.2 The Scale Calibration Principle	5
2.3 Perspective Correction Explained	6
3. Implementation Phases	7
Phase 1: ArUco Detection & Scale Calibration (Week 1)	7
Phase 2: Perspective Correction (Week 1–2)	7
Phase 3: LLM-Based Element Detection (Week 2–3)	8
Phase 4: OpenCV Corner Refinement (Week 3–4)	9
Phase 5: JSON Output & Integration (Week 4–6)	9
4. Surveyor Field Workflow	11
4.1 Equipment Required	11
4.2 Step-by-Step Field Process	11
4.3 Photo Guidelines for Surveyors	11
5. ArUco Marker Specifications & Production	13
5.1 Marker Design Specifications	13
5.2 Production Cost	13
6. Accuracy Analysis	14
6.1 Component-Level Accuracy	14
6.2 End-to-End Accuracy	14
6.3 Accuracy vs. Manufacturing Tolerance	14
7. Final JSON Output Schema	15
7.1 Complete JSON Schema	15
7.2 JSON Field Definitions	16
8. Cost Analysis	18
8.1 Development Cost	18
8.2 Operational Cost (Monthly)	18
8.3 ROI Analysis	18
9. Risks & Mitigation	19
10. Future Roadmap	20
10.1 Short Term (3–6 Months)	20
10.2 Medium Term (6–12 Months)	20
10.3 Long Term (12–24 Months)	20

1. What is an ArUco Marker?

An ArUco marker is a synthetic square marker composed of a wide black border and an inner binary matrix that uniquely identifies it. ArUco markers are part of the OpenCV (Open Source Computer Vision) library and are specifically designed for camera pose estimation, augmented reality, and precise measurement applications.

1.1 Technical Definition

ArUco stands for "Augmented Reality University of Cordoba" — developed by the University of Cordoba, Spain. Each marker consists of a black square border containing a grid of black and white cells that encode a unique binary ID. The most commonly used dictionary is DICT_4X4_50, which provides 50 unique markers in a 4×4 grid pattern.

1.2 Key Properties

Property	Description
Size	Physical: 90×90mm (for Nestup). Can be any size as long as it is known.
Unique ID	Each marker encodes a binary number (0–49 in DICT_4X4_50), enabling multiple markers per scene.
Detection	OpenCV detects markers automatically with sub-pixel accuracy (~0.5px precision).
Robustness	Works under varying lighting, partial occlusion, and moderate rotation/perspective.
Cost	Essentially free — print on any standard printer on card stock.

1.3 Why ArUco for Nestup?

For CNC cabinet manufacturing, site measurements bridge the gap between real-world dimensions and digital fabrication. The ArUco marker provides a known physical reference point in every photograph, enabling automatic scale calibration. This eliminates the guesswork inherent in estimating dimensions from uncalibrated photos and provides a reliable, repeatable measurement pipeline.

2. How the System Works

The ArUco-based measurement system operates as a pipeline that transforms a single photograph into precise dimensional data. The process combines classical computer vision (deterministic, precise) with AI-powered element detection (flexible, intelligent).

2.1 System Architecture Overview

Complete Data Flow

The system processes a wall photograph through five sequential stages, each building on the output of the previous stage:

Step	Stage	Description
1	Image Capture	Surveyor photographs wall with ArUco marker (90×90mm) placed on the wall surface.
2	ArUco Detection	OpenCV automatically detects the marker, extracts 4 corner coordinates with sub-pixel accuracy, and computes the mm-per-pixel scale factor.
3	Perspective Correction	Using the known square geometry of the marker, the system computes a homography matrix to correct perspective distortion across the entire image.
4	AI Element Detection	The original image is sent to Claude API (LLM) which identifies wall elements (doors, windows, outlets, switches) and returns approximate bounding boxes.
5	Corner Refinement	OpenCV edge detection and line intersection algorithms refine the LLM's approximate positions to precise pixel coordinates.
6	Scale Application	The mm-per-pixel scale factor is applied to all refined coordinates, converting pixel positions to real-world millimeter dimensions.
7	JSON Output	Final structured JSON containing wall dimensions, element positions, corner coordinates, and confidence scores.

2.2 The Scale Calibration Principle

The fundamental principle is simple: if you know the real-world size of one object in the photo, you can calculate the real-world size of everything else in the same plane.

Formula: $\text{scale_factor (mm/px)} = \text{marker_known_size_mm} \div \text{marker_detected_size_px}$

Example: If the 90mm ArUco marker appears as 120 pixels wide in the image, then 1 pixel = 0.75mm. A door that spans 2800 pixels wide = $2800 \times 0.75 = 2100\text{mm}$.

This calculation is valid for objects in the same plane as the marker (i.e., the wall surface). Objects at significantly different depths would require additional calibration.

2.3 Perspective Correction Explained

When a surveyor photographs a wall at an angle (which is almost always the case), the image suffers from perspective distortion. A rectangle appears as a trapezoid. Measurements taken directly from distorted pixels will be inaccurate.

The ArUco marker solves this elegantly. Since we know the marker is a perfect 90×90mm square, but it appears as a trapezoid in the image, we can compute the exact mathematical transformation (homography matrix) that maps the distorted image back to a front-facing view. This correction is applied to the entire image, making all pixel measurements geometrically accurate.

3. Implementation Phases

The project is structured into 5 phases over approximately 6–8 weeks. Each phase produces a testable deliverable, allowing progressive validation.

Phase 1: ArUco Detection & Scale Calibration (Week 1)

Objective

Detect the ArUco marker in any wall photograph and compute the mm-per-pixel scale factor with high accuracy.

Technical Implementation

- Set up Python backend with OpenCV and FastAPI
- Implement ArUco marker detection using `cv2.aruco.ArucoDetector`
- Use DICT_4X4_50 dictionary (50 unique markers, 4×4 grid)
- Extract 4 corner coordinates with sub-pixel refinement using `cv2.cornerSubPix`
- Calculate scale factor: $90\text{mm} \div \text{detected_pixel_width}$
- Handle edge cases: marker not found, partial occlusion, multiple markers
- Return scale factor with confidence score

Deliverable

API endpoint that accepts a photo and returns the mm-per-pixel scale factor, marker corner coordinates, and confidence score.

Acceptance Criteria

- Detects marker in 95%+ of well-lit photos taken within 3 meters
- Scale factor accuracy within $\pm 2\%$ when validated against laser measurement
- Processing time under 500ms

Phase 2: Perspective Correction (Week 1–2)

Objective

Use the ArUco marker's known geometry to correct perspective distortion, enabling accurate measurements across the entire wall surface.

Technical Implementation

- Compute homography matrix H from detected marker corners (source) to ideal square corners (destination)

- Apply `cv2.warpPerspective` to transform the entire image to a front-facing view
- Recalculate scale factor on the corrected image for improved accuracy
- Implement quality checks: reject images where perspective distortion exceeds correctable limits (marker angle $> 60^\circ$)
- Handle wall curvature and non-planar surfaces with appropriate warnings

Deliverable

Enhanced API that returns both the original and perspective-corrected image, with updated scale factors for both.

Acceptance Criteria

- Perspective correction reduces measurement error by at least 50% on angled photos
- System correctly rejects uncorrectable images with clear error messages
- Processing time under 1 second including correction

Phase 3: LLM-Based Element Detection (Week 2–3)

Objective

Use Claude API (or equivalent vision LLM) to identify wall elements and their approximate positions from the photograph.

Technical Implementation

- Integrate Claude API with vision capabilities (base64 image upload)
- Design optimized prompt with heuristics: standard door height (2100mm), outlet height (300mm from floor), switch height (1200mm from floor)
- Request structured JSON output with element type, approximate bounding box in pixel coordinates
- Implement prompt engineering for consistent output format
- Add retry logic and fallback handling for API failures
- Cache results to avoid duplicate API calls for the same image

Prompt Strategy

The prompt instructs the LLM to identify elements visible on the wall, classify them by type (door, window, outlet, switch, AC unit, etc.), and provide approximate pixel-coordinate bounding boxes. The LLM also describes any obstructions or ambiguities for surveyor review.

Deliverable

API endpoint that accepts a wall photo and returns a list of detected elements with types and approximate pixel regions.

Acceptance Criteria

- Correctly identifies 90%+ of doors and windows in clear photos
- Correctly identifies 80%+ of outlets and switches
- Bounding boxes are within 50px of actual element boundaries
- Response time under 10 seconds (including LLM API latency)

Phase 4: OpenCV Corner Refinement (Week 3–4)

Objective

Refine the LLM's approximate bounding boxes to precise corner coordinates using classical computer vision techniques.

Technical Implementation

- Crop each element region from the perspective-corrected image (with padding)
- Apply Canny edge detection to find strong edges within each region
- Use Hough Line Transform to detect straight lines (door frames, window edges)
- Calculate line intersections to identify corner points
- Apply `cv2.cornerSubPix` for sub-pixel corner refinement
- Filter outlier corners and validate rectangular geometry
- Convert refined pixel corners to mm coordinates using scale factor

Deliverable

Complete measurement pipeline that combines LLM detection with OpenCV refinement to produce precise corner coordinates and dimensions for each wall element.

Acceptance Criteria

- Corner accuracy within ± 5 –10 pixels on perspective-corrected images
- Dimensional accuracy within ± 2 –5% of actual measurements
- Successfully processes 85%+ of images without manual intervention

Phase 5: JSON Output & Integration (Week 4–6)

Objective

Produce the final structured JSON output and integrate with the Nestup webapp for surveyor use.

Technical Implementation

- Define comprehensive JSON schema covering all output fields
- Implement wall boundary detection (full wall corners, not just elements)
- Calculate relative positions: from_left_mm, from_floor_mm for each element
- Add confidence scores for each measurement (based on edge clarity, marker quality)
- Build surveyor review UI: display detected elements on the photo, allow drag-to-correct
- Implement correction feedback loop: surveyor corrections are logged for future model improvement
- Connect output JSON to existing Nestup design pipeline (SketchUp / CNC workflow)

Deliverable

Production-ready API with surveyor review interface, outputting validated JSON that feeds directly into the Nestup CNC manufacturing pipeline.

Acceptance Criteria

- JSON output passes schema validation 100% of the time
- Surveyor can review and correct measurements in under 2 minutes per wall
- Corrected data is stored for future training data collection
- End-to-end processing time under 15 seconds per wall

4. Surveyor Field Workflow

The success of this system depends on a simple, repeatable field workflow. Surveyors must require minimal training to capture high-quality data consistently.

4.1 Equipment Required

Item	Cost	Notes
Smartphone	Already owned	Any phone with 12MP+ camera. iPhone or modern Android.
ArUco Marker Card	₹10–20 per card	90×90mm marker printed on rigid card stock. Laminated for durability.
Blue Tack / Adhesive	₹50	Reusable adhesive to temporarily stick marker to wall.
Laser Distance Meter	₹3,000–8,000	Bosch GLM series or equivalent. For critical dimension verification.

4.2 Step-by-Step Field Process

- **Step 1 — Prepare:** Enter the room. Identify each wall to be measured. Clear any movable obstructions.
- **Step 2 — Place Marker:** Stick the ArUco marker card on the wall using blue tack. Place it on a flat, unobstructed section of the wall, preferably near the center and at chest height. Ensure the marker is flat against the wall (no curling or tilting).
- **Step 3 — Photograph:** Stand back 2–3 meters from the wall. Ensure the entire wall is visible in the frame, including all doors, windows, and the marker. Take the photo. Avoid extreme angles (try to face the wall as directly as possible).
- **Step 4 — Verify:** Check the photo on screen. Confirm the marker is clearly visible (not blurry, not in shadow). Retake if needed.
- **Step 5 — Laser Verify:** Take 2–3 laser measurements for critical dimensions: overall wall width, floor-to-ceiling height, and any tight-fit area. Enter these into the app.
- **Step 6 — Upload & Review:** Upload the photo via the Nestup webapp. The system processes it and returns detected elements with measurements. Review the overlay, correct any misdetections, and confirm.
- **Step 7 — Remove & Repeat:** Remove the marker from the wall. Move to the next wall. Repeat the process.

4.3 Photo Guidelines for Surveyors

Do	Don't
----	-------

Face the wall as directly as possible	Don't shoot at extreme angles ($> 45^\circ$)
Include entire wall in frame	Don't crop out edges of the wall
Ensure marker is well-lit and sharp	Don't take photos in very low light or against bright windows
Keep phone steady (use both hands)	Don't use zoom (it reduces quality)
Stand 2–3 meters from wall	Don't stand too close (distortion) or too far (marker too small)

5. ArUco Marker Specifications & Production

5.1 Marker Design Specifications

Parameter	Value
Dictionary	DICT_4X4_50 (OpenCV standard)
Marker ID	ID 7 (default; any ID 0–49 works)
Physical Size	90mm × 90mm (outer boundary including black border)
Inner Grid	4×4 binary cells within the border
Border Width	1 cell width (approximately 12.8mm at 90mm total)
White Margin	Minimum 10mm around the marker for reliable detection
Card Total Size	110mm × 130mm (includes margin + Nestup branding)
Print Resolution	Minimum 300 DPI. 600 DPI recommended.
Material	300 GSM card stock, matte laminated (avoids glare)

5.2 Production Cost

Item	Quantity	Cost
Design & Print File Creation	1	Free (generated by OpenCV)
Card Stock Printing (300 GSM)	100 cards	₹1,000–2,000
Matte Lamination	100 cards	₹500–1,000
Total for 100 Markers	100	₹1,500–3,000 (₹15–30 per marker)

Each surveyor should carry 3–5 markers. With a team of 20 surveyors, 100 markers provides adequate supply with spares.

6. Accuracy Analysis

Understanding the accuracy characteristics of each component is critical for setting correct expectations and identifying where to invest in improvement.

6.1 Component-Level Accuracy

Component	Accuracy	Confidence	Notes
ArUco Detection	± 0.5 px	99%+	Sub-pixel refinement
Scale Calibration	± 1 – 2%	95%+	Depends on marker distance
Perspective Correction	± 2 – 3%	90%+	Degrades beyond 45°
LLM Element Detection	$\sim 90\%$ recall	85%+	May miss small elements
Corner Refinement	± 3 – 5 px	90%+	Edge clarity dependent

6.2 End-to-End Accuracy

Measurement Type	Photo Only	Photo + Laser Verify
Wall Width	± 20 – 50 mm	± 1.5 mm (laser)
Wall Height	± 20 – 50 mm	± 1.5 mm (laser)
Door/Window Width	± 10 – 30 mm	± 5 – 10 mm
Element Position	± 15 – 40 mm	± 5 – 15 mm
Outlet/Switch Position	± 20 – 50 mm	± 10 – 20 mm

6.3 Accuracy vs. Manufacturing Tolerance

For CNC cabinet manufacturing, the critical question is whether measurement accuracy is sufficient for the manufacturing process. The answer is yes, with appropriate design margins.

- CNC cutting precision: ± 0.1 mm (machine-limited, not measurement-limited)
- Standard cabinet fitting gap: 2–5mm per side
- Photo-based measurement accuracy: ± 10 – 30 mm (well within gap tolerance)
- Laser-verified critical dimensions: ± 1.5 mm (exceeds requirements)

The hybrid approach (photo for layout + laser for critical dimensions) provides accuracy that exceeds manufacturing requirements while maintaining field speed.

7. Final JSON Output Schema

The system produces a comprehensive JSON document for each wall, containing all measurements, coordinates, and metadata needed to proceed with design and manufacturing.

7.1 Complete JSON Schema

```
{
  "project_id": "NEST-2026-0234",
  "wall_id": "living_room_north",
  "captured_at": "2026-02-10T14:30:00+05:30",
  "surveyor": "Ravi Kumar",
  "image_file": "wall_LR_north_001.jpg",

  "calibration": {
    "aruco_id": 7,
    "aruco_size_mm": 90,
    "aruco_corners_px": [[845,520],[965,518],[967,638],[843,640]],
    "mm_per_pixel": 0.75,
    "perspective_corrected": true,
    "homography_matrix": [[1.02,0.01,-5.2],[0.0,1.01,-3.1],[0.0,0.0,1.0]],
    "detection_confidence": 0.98
  },

  "wall": {
    "width_mm": 4200,
    "height_mm": 3000,
    "width_source": "laser",
    "height_source": "laser",
    "width_accuracy_mm": 1.5,
    "height_accuracy_mm": 1.5,
    "corners_px": [[42,28],[5618,25],[5620,4002],[40,4005]],
    "corners_mm": [[0,0],[4200,0],[4200,3000],[0,3000]]
  },

  "elements": [
    {
      "id": "elem_001",
      "type": "door",
      "subtype": "single_leaf",
      "width_mm": 910,
      "height_mm": 2100,
      "position": {
        "from_left_mm": 500,
        "from_right_mm": 2790,
        "from_floor_mm": 0,
        "from_ceiling_mm": 900
      },
      "corners_px": [[708,1202],[1921,1198],[1923,3998],[706,4002]],
      "corners_mm": [[500,0],[1410,0],[1410,2100],[500,2100]],
      "measurement_source": "photo_refined",
      "accuracy_mm": 15,
      "confidence": 0.92,
      "notes": "Standard wooden door, opens inward"
    },
    {
      "id": "elem_002",
      "type": "window",
      "subtype": "sliding_2panel",
      "width_mm": 1200,
```

```

    "height_mm": 1050,
    "position": {
      "from_left_mm": 2100,
      "from_right_mm": 900,
      "from_floor_mm": 900,
      "from_ceiling_mm": 1050
    },
    "corners_px": [[2802,1335],[4402,1330],[4405,2735],[2800,2740]],
    "corners_mm": [[2100,900],[3300,900],[3300,1950],[2100,1950]],
    "measurement_source": "photo_refined",
    "accuracy_mm": 20,
    "confidence": 0.89,
    "notes": "Sliding window with grill"
  },
  {
    "id": "elem_003",
    "type": "outlet",
    "subtype": "power_socket",
    "width_mm": 70,
    "height_mm": 70,
    "position": {
      "from_left_mm": 1800,
      "from_right_mm": 2330,
      "from_floor_mm": 300,
      "from_ceiling_mm": 2630
    },
    "corners_px": [[2401,3602],[2494,3600],[2496,3695],[2399,3697]],
    "corners_mm": [[1800,300],[1870,300],[1870,370],[1800,370]],
    "measurement_source": "photo_estimated",
    "accuracy_mm": 30,
    "confidence": 0.78,
    "notes": "5A power socket"
  }
],

"metadata": {
  "processing_time_ms": 8500,
  "llm_model": "claude-sonnet-4-5-20250929",
  "opencv_version": "4.9.0",
  "pipeline_version": "1.0.0",
  "surveyor_corrections": 0,
  "laser_measurements_count": 2
}
}

```

7.2 JSON Field Definitions

Field	Description
calibration	ArUco marker detection results: ID, pixel corners, scale factor, homography matrix, and detection confidence.
wall	Overall wall dimensions and corner coordinates. Source indicates whether measurement came from photo or laser.
elements[]	Array of detected wall elements. Each includes type, dimensions, position relative to wall edges, pixel and mm corner coordinates.
corners_px	Corner coordinates in original image pixels [x, y]. Order: top-left, top-right, bottom-right, bottom-left.
corners_mm	Corner coordinates in millimeters relative to wall origin (bottom-left = [0,0]). Same order as corners_px.

measurement_source	"laser" = verified by laser meter, "photo_refined" = OpenCV refined, "photo_estimated" = LLM estimate only.
confidence	0.0–1.0 score indicating system confidence. Below 0.7 triggers surveyor review flag.

8. Cost Analysis

8.1 Development Cost

Item	Duration	Estimated Cost
Phase 1: ArUco Detection	1 week	Internal development
Phase 2: Perspective Correction	1 week	Internal development
Phase 3: LLM Integration	1–2 weeks	Internal + API testing costs
Phase 4: Corner Refinement	1–2 weeks	Internal development
Phase 5: Integration & UI	2 weeks	Internal development
Total Development	6–8 weeks	Primarily internal dev time

8.2 Operational Cost (Monthly)

Item	Volume	Monthly Cost
Claude API Calls	500 walls/month	₹2,500–5,000
Server Hosting (Python backend)	1 instance	₹1,500–3,000
Marker Replacement	20 markers/month	₹300–600
Total Monthly		₹4,300–8,600

8.3 ROI Analysis

Current manual measurement process: each wall takes approximately 15–20 minutes for a surveyor to measure manually with a tape measure, record dimensions, and sketch the layout.

With ArUco system: each wall takes approximately 3–5 minutes (photo + laser verify + review). This represents a 75–80% reduction in surveyor time per wall.

Metric	Current (Manual)	With ArUco System
Time per wall	15–20 min	3–5 min
Walls per day (per surveyor)	15–20	50–70
Measurement errors	5–10%	1–3%
Data format	Paper sketch	Structured JSON
CNC pipeline integration	Manual data entry	Automatic

9. Risks & Mitigation

Risk	Severity	Likelihood	Mitigation
Marker not detected (poor lighting, blur)	Medium	15–20% of photos	Real-time detection feedback in app. Retake prompt before upload.
LLM misidentifies elements	Medium	10–15%	Mandatory surveyor review step. Corrections logged for improvement.
Extreme perspective angle	Low	5–10%	Angle detection with warning. Reject images beyond 60°.
Marker physically damaged	Low	5%	Surveyors carry 3–5 spare markers. Low replacement cost.
Claude API downtime	Medium	Rare	Queue photos for processing when API recovers. Offline mode captures only.
Surveyor adoption resistance	High	Moderate	Training sessions. Show time savings. Start with tech-forward surveyors.
Non-planar walls (curved, multi-plane)	Low	5%	Flag for manual measurement. Multiple markers for complex walls.

10. Future Roadmap

10.1 Short Term (3–6 Months)

- Launch MVP with Phase 1–5 complete
- Collect surveyor correction data to build training dataset organically
- Optimize prompts based on real-world failure patterns
- Add support for multiple markers per wall (large walls, complex geometry)
- Develop offline photo capture mode with batch upload

10.2 Medium Term (6–12 Months)

- Train custom ML model on collected correction data (target: 2,000+ labeled images)
- Replace LLM for element detection with custom model (faster, cheaper, more consistent)
- Add room-level stitching: combine multiple wall JSONs into complete room model
- Integrate with SketchUp plugin for automatic 3D room generation from JSON
- Develop automated quality scoring for photos (reject poor quality before processing)

10.3 Long Term (12–24 Months)

- Full room scanning from video walkthrough (frame extraction + stitching)
- Integration with iPhone/iPad LiDAR for hybrid photo + depth measurement
- Automated design suggestion engine: JSON input → cabinet layout proposals
- White-label the measurement system for other interior design companies
- Complete automated pipeline: site photo → room model → cabinet design → G-code → CNC cutting

End of Document

Nestup Tech — Confidential